0. **Code:** sotomayorj1594 Sotomayorj1594 Test02.zip

1. Array 2 Nest Loop/Program.cs
2. Binary Search Positive Root/Program.cs
3. Test02 Array 1 Larger Running Average/Program.cs

1. Array 2 Nest Loop/Program.cs

```
using System.Collections;

namespace Array_2_Nest_Loop
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int[] arrayTest = {
                7,
                3,
                1,
                8,
                2,
                4,
                5,
                0,
                9
            };
            Console.WriteLine("The final array: " + string.Join(" ",
              excludeSmallValue(arrayTest)));
        }
        //1 create method that accepts array.
        static int[] excludeSmallValue(int[] originalA)
        {
            if (originalA == null)
            {
                return null;
            }
            if (originalA.Length == 0)
            {
```

B1

1. **Array 2 Nest Loop/Program.cs**

```
            return originalA;
        }
        //2 new array with length from calc
        int lengthA = (originalA.Length * (originalA.Length + 1)) /
          2;
        int[] newA = new int[lengthA];
        int min = int.MaxValue;
        //3 copy all vlaues from org to new
        for (int i = 0; i < originalA.Length; i++)
        {
            newA[i] = originalA[i];

        }
        //4 exclude smallest value into new array
        int j = 0;
        for (int rep = 0; rep < lengthA; rep++)
        {
            /*if (newA[i] < min)
            {
                min = newA[i];
            }*/
            for (int i = 0; i < originalA.Length; i++)
            {
                if (originalA[i] > rep) //not (originalA[i] > rep) use min
                {
                    newA[j] = originalA[i];
                    j++;

                }
            }
        }
```

B2

B5 X Irrelevant Logic.

needs to nest inside another Loop

B4

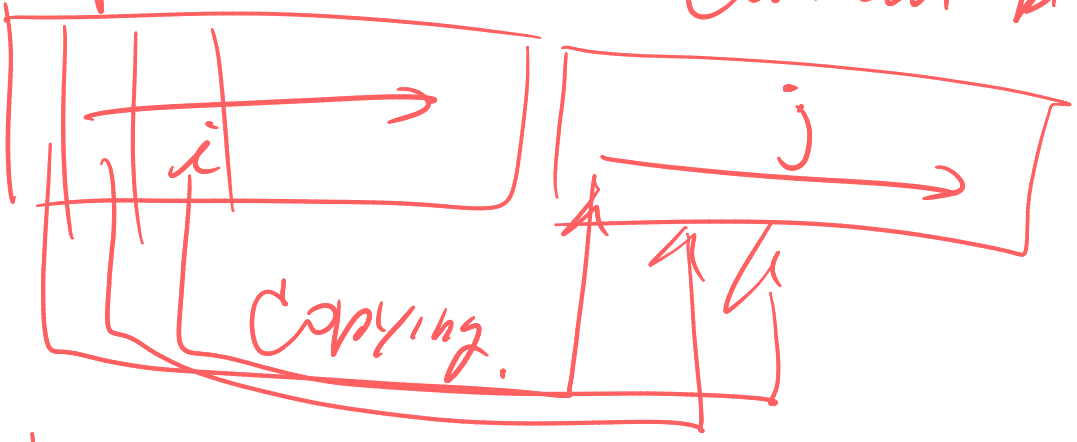
B3

1. **Array 2 Nest Loop/Program.cs**

```
            return newA;
        }
    }
}
```

(rep-1)th iteration. "rep th iteration

previous block current block



You need to keep track of additional index.

2. Binary Search Positive Root/Program.cs

```
namespace Binary_Search_Positive_Root
{
    internal class Program
    {
        static void Main(string[] args)
        {

            Console.WriteLine("Hello World");

        }

        //1 create funcion named f
        static int f(int x, int b, int c)
        { //2 calculate equation
            int FofX = (x * x) + (b * x) - c;
            return FofX;
        }
    }
}
```

ok C1

C2

3. Test02 Array 1 Larger Running Average/Program.cs

```
using System.Collections;

namespace Test02_Array_1_Larger_Running_Average
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int[] arrayTest = {
                7,
                3,
                1,
                8,
                2,
                4,
                5,
                0,
                9
            };
            Console.WriteLine(LRA(arrayTest));

        }
        //1 arg that accepts int array
        static double LRA(int[] arr)
        {
            if (arr == null || arr.Length == 0 || arr.Length == 1)
            {
                return 0;
            }
            //2 calc and return running avg
            double sum = 0;
```

A1

A3

3. Test02 Array 1 Larger Running Average/Program.cs

```
            int counter = 0;
            double runningAvg = 0;
            double finalAvg = 0;
            for (int index = 0; index < arr.Length; index++)
            {
                int item = arr[index];
                double prevSum = sum;

                sum = sum + item;
                if (index > 0)
                {
                    runningAvg = prevSum / index;

                    if (item > runningAvg)
                    {
                        finalAvg = finalAvg + item;
                        counter++;
                    }
                }
            }
            if (counter != 0)
            {
                double largerRunning = finalAvg / counter;
                return largerRunning;
            }
            else
            { //if no values reutrn zero
                return 0;
            }

        }
```

A2c

A2b

A2

A3

3. Test02 Array 1 Larger Running Average/Program.cs

```
    }
}
```